

P1 Project - Final Report

Multi-View Clustering for Localized Global Time Series Forecasting - Applied to Electricity Consumption Data

Submitted By: Wishal Fatima
University of Vienna | June 2026

1. Introduction

This report documents the complete work carried out during the P1 project, titled 'Multi-view clustering for localized global time series forecasting'.

Phase 1 focused on understanding the theoretical foundations of the OFMVC algorithm by running experiments on standard benchmark datasets from the UCI repository. Phase 2 applied the algorithm to a real-world electricity consumption dataset provided by the supervisor, with multiple rounds of improvement based on the feedback.

The core research question was: can multi-view clustering grouping electricity customers by consumption patterns across multiple data views (smartmeter data and weather information) improve forecasting accuracy compared to simpler clustering approaches or no clustering at all?

This report also documents a significant implementation finding: a critical bug was identified in the reference implementation provided by the paper's authors, where the key algorithmic feature of OFMVC the view-weight optimization step was disabled by default. This is not immediately obvious because the algorithm still runs completely without errors, produces valid-looking cluster assignments, and prints convergence values at each iteration giving no indication that anything is wrong. The discovery, diagnosis, and correction of this bug is a central contribution of this project, and it fundamentally changes the comparative results.

2. Background: The OFMVC Paper

2.1 Paper Overview

This project is based on the following publication:

Abbasimehr, H., Abri, A., Golzari Oskouei, A., & Noshad, A. (2025). Multi-view clustering for localized global time series forecasting. *Neurocomputing*, 653, 131183. <https://doi.org/10.1016/j.neucom.2025.131183>

The paper proposes a new forecasting methodology that combines multi-view clustering with time series imaging techniques to improve the accuracy of global forecasting models. The

source code is publicly available at: <https://github.com/AliReza000J/MVC-Time-series-Forecasting>

2.2 Problem Being Solved

Time series forecasting has traditionally relied on either local models (one model per series) or global models (one model trained across all series). Both have limitations:

- Local models (ETS, ARIMA): cannot leverage patterns shared across similar series. Each series is treated in isolation, which is inefficient when many series share common behavior.
- Global Forecasting Models (GFMs): trained on all series simultaneously, capturing shared patterns. However, when a dataset contains heterogeneous series (e.g., customers with very different consumption behaviors), a single global model performs poorly for individual series.

The paper proposes a middle path: cluster similar time series together, then train a separate global model per cluster. This 'localized global' approach captures shared patterns within each homogeneous group while remaining flexible across different groups. The central challenge is how to perform this clustering effectively when no ground-truth labels exist to evaluate clustering quality.

2.3 Multi-View Representation

The paper's key innovation is the use of time series imaging techniques to generate multiple complementary views of each time series, rather than relying on manually crafted features. Three imaging techniques are used:

- Recurrence Plot (RP): encodes the recurrence of states in the phase space of the time series. It captures periodic, chaotic, and non-stationary behaviors that are not visible in the raw series.
- Gramian Angular Field (GAF): converts the time series to a polar coordinate representation and encodes temporal correlations as a Gram matrix. It captures angular relationships between time steps.
- Markov Transition Field (MTF): encodes the probability of transitions between quantile bins over time. It captures the dynamic statistics and transition patterns of the series.

In the original paper, each time series is converted into three different image representations. These images are then encoded into compact latent vectors using a TCN autoencoder, producing three views per time series. In this project, the weather time series replace these imaging techniques as the additional views, providing naturally occurring complementary information rather than mathematical transformations.

2.4 The OFMVC Algorithm

OFMVC (Optimal Fuzzy Multi-View Clustering) is proposed as the clustering algorithm that operates on this multi-view dataset. It extends FMVC (Fuzzy Multi-View Clustering, proposed in prior work) by adding two key mechanisms:

- View-weight optimization: OFMVC learns a weight for each view, reflecting how informative that view is for clustering. Views that better capture the intrinsic structure of the data receive higher weights. This is the 'Optimal' in OFMVC, the algorithm optimally determines the contribution of each view, rather than treating all views uniformly as FMVC does.
- Manifold and diversity constraints: OFMVC incorporates neighborhood information (manifold learning) to preserve the local geometric structure of the data. A diversity constraint also ensures that the selected views are complementary to each other, rather than redundant. These are controlled by parameters α_1 (view diversity) and NR (neighborhood size).

The algorithm iterates between four update steps until convergence:

1. Update cluster membership assignments for each customer in each view
2. Update cluster centers (centroids) for each view
3. Update feature weights within each view (which features/days matter most)
4. Update view weights (which views matter most overall) (This differentiates OFMVC from FMVC)

FMVC performs only steps 1–3, keeping view weights fixed and uniform throughout. Step 4 is the critical differentiator.

Objective Function

"The core OFMVC objective function being minimized is:

$$J = \sum_v w_v^\beta \sum_i \sum_j u_{ij}^m d_{ij}^v + \frac{\alpha_1}{NR} \sum_v \sum_i \sum_j u_{ij}^m ||x_i^v - N_i^v||^2$$

Where:

- w_v = weight of view v (optimized in Step 4 by OFMVC)
- u_{ij} = fuzzy membership of customer i in cluster j
- m = fuzzy exponent (= 2 in this project)
- x_{iv} = data point i in view v
- c_{jv} = cluster center j in view v
- N_{iv} = neighborhood of customer i in view v
- α_1 = diversity parameter (= 0.5)
- NR = neighborhood size (= 5)

FMVC minimizes the same function but with w_v fixed at $1/n_views$ throughout — never updated.

The paper itself uses FMVC as the primary baseline for comparison against OFMVC, the two algorithms are identical except that FMVC keeps view weights fixed and uniform while OFMVC optimizes them. This distinction is central to all comparisons in this project.

Table 1: Overview of Clustering Algorithms Used in This Project

Aspect	OFMVC	FMVC	K-Means	kShape
View handling	Each view processed separately with learned weights	Each view processed separately, uniform weights	All views concatenated into one matrix	Single view only (no weather)
View weight learning	Yes — learns optimal weight per view (Step 4)	No — all views weighted equally at $1/n_views$	Not applicable	Not applicable
Neighborhood information	Yes — manifold constraint using k-nearest neighbors	No	No	No
Fuzzy memberships	Yes — each customer has soft membership in all clusters	Yes — same fuzzy approach	No — hard assignment only	No — hard assignment only
Distance metric	Exponential kernel distance per view	Exponential kernel distance per view	Euclidean distance in concatenated space	Normalized cross-correlation (shape-based)
Views used in this project	6 views: smartmeter + 5 weather series	6 views: smartmeter + 5 weather series	6 views concatenated (5000 x 2190 features)	1 view: smartmeter only
Convergence (this project)	6 iterations	6 iterations (uniform weights)	17-32 iterations (10 random inits)	6 iterations

2.5 Original Paper Results

The paper evaluates OFMVC on six M3 monthly forecasting competition datasets. Key findings from the paper:

- OFMVC outperforms all single-view clustering methods on all six datasets.
- In terms of mean sMAPE, OFMVC achieves a 0.21 improvement over FMVC (the baseline multi-view method).
- OFMVC outperforms state-of-the-art models including FFORMA, CatBoost, N-BEATS, DeepAR, and graph-based global forecasting models.

2.6 How this Project adapts the Paper

The original paper generates multiple views using time series imaging techniques applied to a single univariate series. This project uses naturally available external views. Therefore, the goal is not to reproduce the exact representation learning pipeline, but to evaluate whether multi-view clustering improves forecasting when additional contextual information is available. In this project, the views are instead the raw smartmeter consumption data and five naturally-occurring weather time series (temperature and sun radiation). This adaptation is justified because:

- Weather data provides genuinely complementary information about the external drivers of electricity consumption, directly analogous to how RP, GAF, and MTF capture different structural properties.
- The use of weather data alongside the smartmeter data.
- Using real-world contextual views (weather) rather than mathematical transformations (images) makes the clustering results more interpretable, cluster assignments directly reflect how customer groups respond differently to weather conditions.

2.7 Dataset Difference from the Original Paper

The original paper evaluates OFMVC on monthly time series from the M3 forecasting competition, a standard benchmark dataset with no weather information. This project adapts the method to daily electricity consumption forecasting using real smart meter data. Key differences:

- Original paper: monthly data, M3 competition datasets, views generated via imaging techniques
- This project: daily data (365 days), real electricity customers, views are natural weather time series
- Clustering is performed only on 2023 data. 2024 data is entirely reserved for forecasting evaluation and never seen during clustering

3. Phase 1: Familiarization with OFMVC on UCI Datasets

3.1 Objective

Before applying OFMVC to the electricity dataset, the algorithm was first explored on four standard UCI multi-variate time series datasets. The primary goals of this phase included

- Understanding the codebase and verify it runs correctly
- Study how the number of views affects clustering performance
- Answering the research question: Does adding more views always improve performance, or is there a point of diminishing returns or degradation?

In these datasets, each sensor channel or signal dimension was treated as one view. This led us to create multiple views from a time series as described in paper.

3.2 Datasets Used

Dataset	Full Name	Subjects	Activities	Sensor Views	Description
WISDM	Smartphone and Smartwatch Activity and Biometrics	51	18	Accelerometer and gyroscope channels	Physical activity data collected from smartphone and smartwatch sensors during daily activities
HAR	Human Activity Recognition using Smartphones	30	6	3-axis accelerometer + gyroscope	Activities recorded while subjects wore smartphones on their waist
MHEALTH	Mobile Health Dataset	10	12	Body-worn sensor channels (ECG, acceleration, gyroscope)	Physiological and motion signals from subjects performing physical activities
PAMAP2	Physical Activity Monitoring	9	12	IMU sensor channels from 3 body positions	Data from inertial measurement units placed on hand, chest, and ankle

3.3 Algorithm Configuration for UCI Datasets

The OFMVC algorithm was applied using the default parameters from the paper, with the number of views varied to study the effect:

Parameter	Value	Description
k (clusters)	5	Number of target clusters — consistent with paper's experiments
t_max	20	Maximum number of optimization iterations
NR	5	Size of neighborhood for manifold constraint
alpha1	0.5	View diversity parameter — controls complementarity penalty
lambda (landa)	0.5	Regularization strength for feature weighting
q	2	Fuzzy exponent — controls softness of cluster memberships
gamma (gama)	1	Neighborhood weight — controls influence of neighbors
Views tested	1 to max	Incrementally added one view at a time to study effect

3.4 Experimental Results

WISDM Dataset

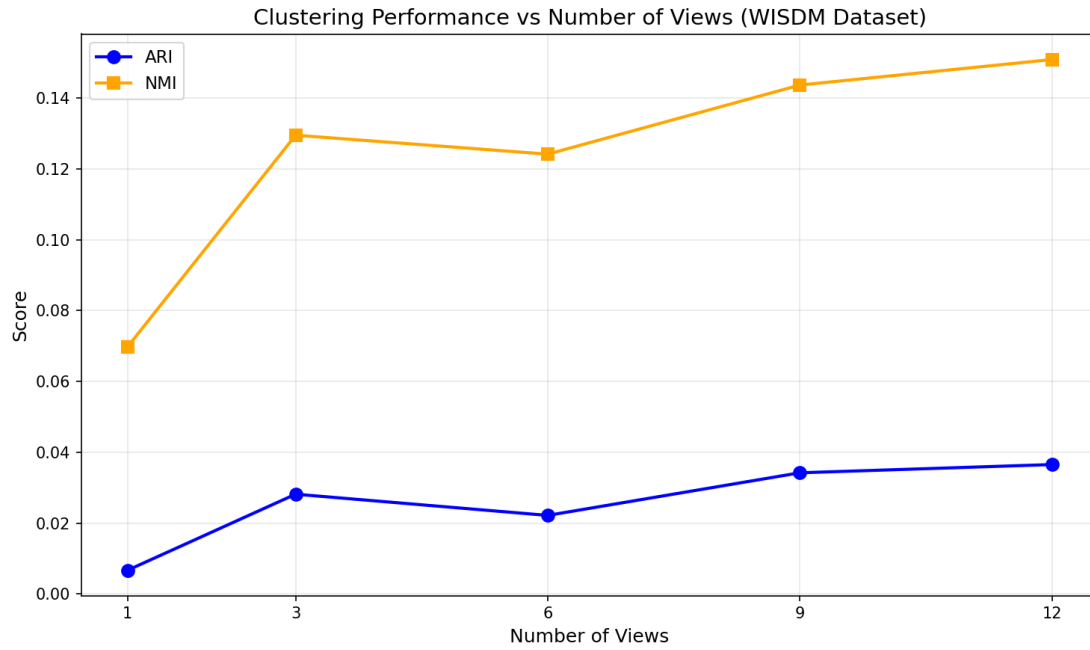


Figure 1: OFMVC clustering performance on the WISDM dataset as the number of views increases. Each point represents adding one additional sensor channel as a view.

The WISDM results show an initial improvement in clustering performance as the first few views are added. Performance then plateaus and shows slight degradation when too many views are included, suggesting that some sensor channels contain redundant or noisy information.

HAR Dataset

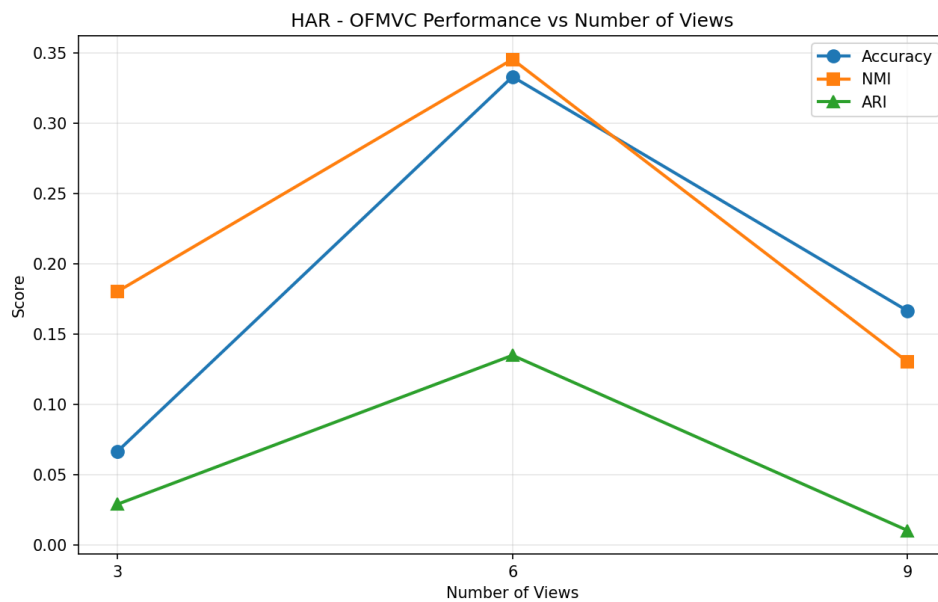


Figure 2: OFMVC clustering performance on the HAR dataset across different numbers of views.

HAR shows a clearer improvement pattern with the first 3–5 views, followed by degradation with additional views. This is consistent with the theoretical expectation that adding too many views introduces noise that confuses the clustering objective.

MHEALTH Dataset

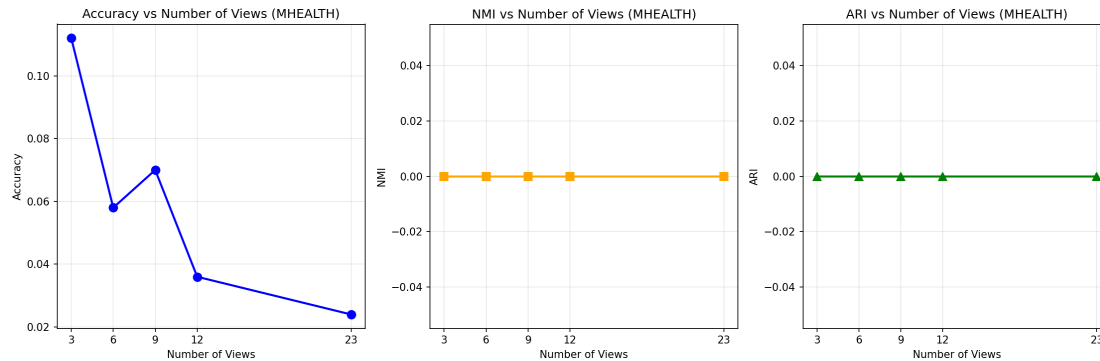


Figure 3: OFMVC clustering performance on the MHEALTH dataset across different numbers of views.

MHEALTH shows a more stable pattern across views, with gradual improvement. The dataset's richer sensor modalities (ECG, acceleration, gyroscope from multiple body positions) appear to provide more complementary information per view.

PAMAP2 Dataset

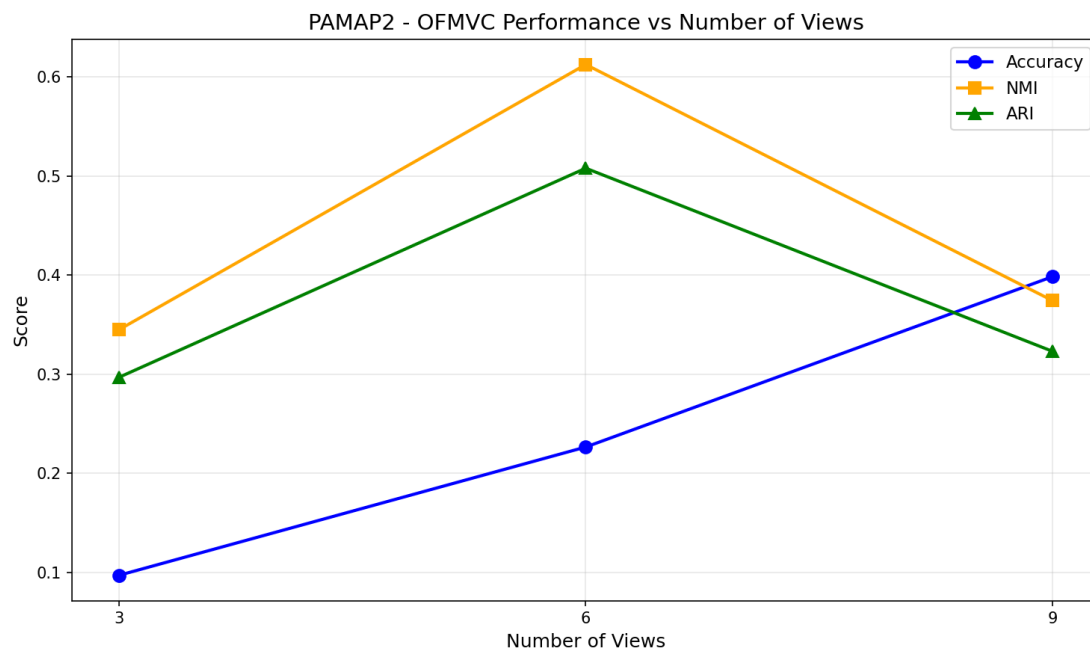


Figure 4: OFMVC clustering performance on the PAMAP2 dataset across different numbers of views. PAMAP2 achieved the best overall performance at 39.9% accuracy with 9 views.

PAMAP2 achieved the best overall performance across all datasets, reaching 39.9% clustering accuracy at 9 views. The three-position IMU setup (hand, chest, ankle) provides genuinely

complementary views of physical activity, explaining why more views continue to help up to a higher count than other datasets.

3.5 Key Findings

Finding	Observation	Implication for Phase 2
Adding views helps up to a point	Performance improved in all datasets when adding the first 3–6 views	Use all 6 available views (smartmeter + 5 weather) but also test subsets
Too many views can hurt	WISDM and HAR showed degradation beyond 6–9 views	Conduct view redundancy analysis to check if any weather views are harmful
Best performance: PAMAP2	39.9% accuracy at 9 views	Datasets with genuinely complementary views benefit most from multi-view approach
View selection matters	Not all views are equally useful	OFMVC's view-weight learning is important — it should down-weight less useful views
Code verification	Algorithm ran successfully on all 4 datasets	Ready to apply to electricity dataset

4. Phase 2: Electricity Consumption Dataset

4.1 Dataset Description

The electricity dataset provided consisted of a 20% sample of real smart meter electricity consumption data from 35,095 customers in Austria, covering two full years: 2023 (used for clustering and model training) and 2024 (used for forecasting evaluation).

The dataset comprises 12 CSV files organized by data type and year:

File	Content	Shape (2023)	Role
sample_23_20percent.csv	Daily electricity consumption per customer	35,095 x 366	Primary view for clustering + forecasting training
sample_24_20percent.csv	Daily electricity consumption per customer	35,095 x 367	Forecasting evaluation (test set)
sample_avgtemp_23/24.csv	Average daily temperature	35,095 x 367	Weather view for clustering
sample_maxtemp_23/24.csv	Maximum daily temperature	35,095 x 367	Weather view for clustering
sample_mintemp_23/24.csv	Minimum daily temperature	35,095 x 367	Weather view for clustering
sample_maxGSun_23/24.csv	Maximum global solar radiation	35,095 x 367	Weather view for clustering
sample_sumGSun_23/24.csv	Sum of global solar radiation per day	35,095 x 367	Weather view for clustering

A subset of 5,000 customers was selected for all experiments. This decision was driven by the computational complexity of OFMVC: constructing the $n \times n$ distance matrices required by the neighborhood-based manifold learning step scales as $O(n^2)$ in both time and memory. Testing confirmed that 50 customers produced poor clustering diversity (only 2 active clusters), 500 customers improved quality substantially, and 5,000 customers produced the most meaningful and stable cluster structure while remaining computationally feasible within 30–60 minutes per run.

4.2 Evaluation Methodology

A fundamental challenge in this project was the absence of ground-truth cluster labels. There is no 'correct' clustering of electricity customers that we can compare against. The paper's solution, adopted directly in this project is to evaluate clustering quality indirectly through downstream forecasting accuracy. The reasoning is: if a clustering is good (i.e., customers in the same cluster genuinely behave similarly), then a single forecasting model trained on that cluster will predict well for all members. If the clustering is poor, the mixed behavior within clusters will produce poor forecasts.

The evaluation pipeline used in this project is as follow:

1. Cluster 5,000 customers using 2023 data and the clustering algorithm under evaluation
2. For each cluster, train a separate TCN (Temporal Convolutional Network) forecasting model using the 2023 smartmeter consumption data of customers in that cluster
3. Use the trained cluster models to predict 2024 electricity consumption
4. Evaluate forecast accuracy using Symmetric Mean Absolute Percentage Error (SMAPE)

The same TCN forecasting pipeline was used for all clustering methods, ensuring that differences in SMAPE reflect only the quality of the clustering, not differences in the forecasting model. This pipeline is adopted directly from the OFMVC codebase.

Forecasting Parameter	Value	Explanation
Model architecture	TCN (Temporal Convolutional Network)	Deep learning model for sequence prediction, from the original OFMVC codebase
Autoencoder type	TCN autoencoder	Encodes time series to latent space before forecasting
Latent dimension	16	Compact representation size
Training data	2023 smartmeter consumption (raw, unscaled)	Raw data required by the Holt-Winters decomposition step
Test data	2024 smartmeter consumption	Held-out year for evaluation
Look-back window (lag)	28 days	Model uses past 28 days to predict
Forecast horizon	18 days	Predicts 18 days ahead
Sample overlap	17 days	Sliding window overlap between training samples
Epochs	10	Training iterations
Batch size	20	Samples per gradient update
Learning rate	0.0001	Adam optimizer learning rate
Evaluation metric	SMAPE (%)	Symmetric Mean Absolute Percentage Error — lower is better

5. Original Implementation and Results

5.1 Data Preprocessing

The original preprocessing pipeline addressed several structural issues in the dataset before clustering could proceed:

5.1.1 Column Mismatch Between Views

The weather datasets contained an additional column labeled '2022-12-31 23:59:59' that was absent from the smartmeter data. This extra column caused dimensionality inconsistencies that would crash the multi-view clustering algorithms. The fix was to drop this extra column from all five weather files before processing:

if '2022-12-31 23:59:59' in df.columns:

```
df.drop(columns=['2022-12-31 23:59:59'], inplace=True)
```

5.1.2 Inconsistent Date Column Naming

The smartmeter dataset used date formats like '2023-01-01' for column names, while the weather datasets used full timestamp formats like '2023-01-01 23:59:59'. This prevented direct column alignment between views. The fix was to rename all weather dataset columns to match the smartmeter date format:

```
new_cols = ['ID'] + list(smart.columns[1:])
```

```
df.columns = new_cols
```

5.1.3 Missing and Invalid Values

The datasets contained missing (NaN) and invalid (Inf) values scattered throughout. These caused numerical errors during OFMVC clustering, particularly during distance matrix computation and center updates. The original approach was to replace all invalid values with zeros:

```
data = np.nan_to_num(data, nan=0.0, posinf=0.0, neginf=0.0)
```

While this allowed the pipeline to run without crashing, replacing temperature values with zero introduced a fundamental data quality problem: a zero temperature reading on a summer day in Austria is factually incorrect and creates artificial patterns in the data. This was later identified as a key limitation and corrected in the improved version.

5.1.4 Normalization

All views were normalized to the range [0,1] using scikit-learn's MinMaxScaler, applied globally across all customers:

```
scaler = MinMaxScaler()
```

```
view_norm = scaler.fit_transform(view)
```

This global scaling approach was also later identified as problematic and replaced. With Min-Max scaling, a customer with unusually high consumption (e.g., 1,051 kWh on a single day) dominates the scale, compressing all other customers' data into a very narrow fraction of the [0,1] range. This makes shape-based pattern comparison unreliable.

5.2 Clustering Algorithms Implemented

Four clustering algorithms were implemented and evaluated. The OFMVC algorithm was taken from the paper's GitHub repository; the remaining three (FMVC, K-Means, kShape) were implemented independently following standard algorithm descriptions and library documentation.

5.2.1 OFMVC Clustering

OFMVC was applied using all 6 views (smartmeter + 5 weather series), processed separately as per the algorithm's design. The OFMVC code was taken directly from the paper's GitHub repository. Parameters followed the paper's recommendations:

Parameter	Value	Purpose
k (clusters)	5	Number of customer groups
t_max	20	Maximum optimization iterations
NR	5	Neighborhood size for manifold constraint
alpha1	0.5	View diversity penalty strength
landa	0.5	Feature regularization strength
q	2	Fuzzy membership exponent
gama	1	Neighborhood influence weight

5.2.2 FMVC Clustering

FMVC (Fuzzy Multi-View Clustering) was implemented from scratch as a vectorized Python implementation. Unlike OFMVC, FMVC does not use neighborhood information or learn view weights, it applies fuzzy c-means clustering principles across multiple concatenated views. The implementation used vectorized NumPy operations for efficient distance computation across 5,000 customers:

```
# Compute distances from each sample to each center
```

```
dist = np.zeros((n_samples, n_clusters))
```

```
for j in range(n_clusters):
```

```
    diff = data - centers[j]
```

```
    dist[:, j] = np.sqrt(np.sum(diff ** 2, axis=1))
```

```
# Update fuzzy memberships
```

```
for j in range(n_clusters):
```

```
    ratio = dist / np.maximum(dist[:, j:j+1], 1e-10)
```

```
    ratio = ratio ** (2 / (m - 1))
```

```
    membership[:, j] = 1 / np.sum(ratio, axis=1)
```

Key implementation decisions:

- A minimum distance threshold of $1e-10$ was used to prevent division by zero in the membership update
- When a sample's distance to a center was exactly zero, it was assigned full membership to that cluster
- Convergence was checked by measuring the maximum change in the membership matrix across iterations, with a threshold of $\epsilon = 1e-6$.

Parameter	Value
n_clusters	5
fuzzy_exponent (m)	2
max_iter	30
convergence threshold (epsilon)	$1e-6$
Views used	6 views concatenated (5000 x 2190 features)

5.2.3 K-Means Clustering

K-Means clustering was implemented using scikit-learn's KMeans class. To enable fair comparison with OFMVC and FMVC (which use all 6 views), K-Means used all six views concatenated into a single feature matrix of shape 5,000 x 2,190 (6 views x 365 days each). This concatenation approach is the standard adaptation for applying K-Means to multi-view data.

```
# Concatenate all 6 normalized views into one feature matrix
```

```
data = np.hstack(normalized_views) # shape: (5000, 2190)
```

```
kmeans = KMeans(  
    n_clusters=n_clusters,  
    random_state=42,  
    n_init=10,      # 10 random initializations, best result selected  
    max_iter=300,  # maximum iterations per run  
    verbose=1  
)  
cluster_labels = kmeans.fit_predict(data)
```

Using `n_init=10` (10 independent random initializations with the best result selected) was important for K-Means stability, since K-Means is sensitive to initialization and can converge to poor local minima. The best clustering was selected based on inertia (within-cluster sum of squares).

5.2.4 kShape Clustering

Unlike OFMVC and FMVC, kShape uses only the smartmeter consumption view. It is a single-view algorithm and cannot incorporate weather data. This makes kShape a natural baseline for the single-view case.

```
from tslearn.clustering import KShape
```

```
# Normalize data (Min-Max row-wise for kShape)
```

```
data_normalized = scaler.fit_transform(data.T).T
```

```
# Reshape for kShape: (n_samples, n_timestamps, n_features=1)
```

```
data_reshaped = data_normalized.reshape(n_customers, n_days, 1)
```

```
kshape = KShape(n_clusters=5, max_iter=30, random_state=42, verbose=True)
```

```
cluster_labels = kshape.fit_predict(data_reshaped)
```

kShape was implemented using tslearn's KShape class. Since kShape is a shape-based single-view algorithm, only the smartmeter consumption data was used, making it a natural single-view baseline. Data was Min-Max normalized and reshaped to (5000, 365, 1) as required by tslearn. The algorithm converged after 6 iterations producing all 5 active clusters with distribution [494, 2236, 37, 444, 1789].

5.3 Original Clustering Results

Algorithm	Views Used	Active Clusters	Cluster Distribution	Convergence
OFMVC	6 separate views	4 out of 5	1529, 208, 742, 2521	6 iterations
FMVC (author implementation)	6 concatenated views	3 out of 5	888, 2721, 1391	30 iterations (unstable — did not converge)
K-Means (author implementation)	6 concatenated views	5 out of 5	991, 1779, 468, 186, 1576	17 to 32 iterations across 10 random runs

Algorithm	Views Used	Active Clusters	Cluster Distribution	Convergence
kShape (author implementation)	1 view (smartmeter)	5 out of 5	494, 2236, 37, 444, 1789	6 iterations

Notable observations from the original clustering:

- OFMVC and FMVC both produced one empty cluster (Cluster 0), suggesting that with 5,000 customers and 5 target clusters, one cluster consistently fails to attract any customers during initialization. This may indicate that 5 clusters is slightly more than the natural grouping structure of this dataset.
- FMVC was notably unstable, it did not converge within 30 iterations. The membership update changes initially decreased but later increased and plateaued around 0.35, suggesting the algorithm oscillated rather than converging. This instability is likely due to the high dimensionality of the concatenated feature space (5,000 x 2,190) combined with the non-separable nature of the data in that space.
- kShape produced all 5 clusters but with a highly imbalanced distribution, Cluster 2 contained only 37 customers (0.7%), indicating a very specific and rare consumption pattern in the dataset.
- K-Means was the most stable in terms of all clusters being populated and having relatively balanced sizes.

5.4 Original Forecasting Results

Algorithm	Test SMAPE	Validation SMAPE	Rank	Notes
K-Means	30.84%	29.73%	1st	Best result — unexpected given its simplicity
FMVC	30.89%	30.36%	2nd	Very close to K-Means
OFMVC	31.46%	31.18%	3rd	Should be best
kShape	31.57%	30.26%	4th	Single-view baseline — expected to be lower

The result that K-Means (30.84%) outperformed OFMVC (31.46%) was flagged as surprising. K-Means is a simple, general-purpose algorithm with no time series-specific features, while OFMVC is a sophisticated algorithm specifically designed for multi-view time series clustering with view-weight optimization and manifold constraints. The gap of 0.62 percentage points in favor of K-Means was unexpected and prompted a deeper investigation into the implementation, which led to the discovery described below.

6. Improvements and Corrections

6.1 Preprocessing Improvement: Interpolation Instead of Zero-Fill

The original approach of replacing all NaN values with zeros was identified as a significant data quality problem. For electricity consumption data, a zero value means no electricity was consumed, which is a valid real-world value. For weather data, a zero temperature reading on a summer day in Austria is physically implausible and would mislead the clustering algorithm into treating these artificial readings as genuine cold periods.

An alternative approach of simply dropping all rows (customers) with any missing values was considered first. However, this reduced the dataset from 35,095 customers to only 821, a 97.7% reduction that would make all statistical conclusions unreliable and was explicitly flagged by the supervisor as too few samples.

The chosen solution was linear interpolation with forward and backward fill, implemented in a custom preprocessing script written for this project. The key function:

```
def interpolate_and_z_norm(df, is_weather=False):

    # Align to smartmeter date columns

    keep_cols = ['ID'] + [c for c in df.columns[1:] if c in date_cols
                           or c.split()[0] in date_cols]

    df_aligned = df[keep_cols]

    ids = df_aligned['ID']

    data = df_aligned.drop(columns=['ID'])

    # Interpolate across days (row-wise) — linear with both-direction fill

    data = data.interpolate(method='linear', axis=1, limit_direction='both')

    data = data.fillna(0) # safety fallback for any edge NaNs

    # Z-normalize each customer independently

    def z_norm(row):

        mean = row.mean()

        std = row.std()

        if std == 0:

            return row - mean

        return (row - mean) / std
```

```
data_norm = data.apply(z_norm, axis=1)

return pd.concat([ids, data_norm], axis=1)
```

This approach fills gaps in each customer's time series by linearly interpolating between known neighboring values, producing smooth, physically plausible values rather than artificial zeros. The `limit_direction='both'` parameter ensures that even values at the very beginning or end of a series are filled (not just interior gaps).

Result: All 5,000 customers were retained with zero artificial zero-values in the weather data. The interpolated data preserved the natural seasonal and daily patterns of each weather variable.

6.2 Normalization Improvement: Z-Normalization per Customer

The original global Min-Max normalization was replaced with per-customer Z-normalization. The problem with global Min-Max normalization is illustrated by a concrete example from this dataset: the maximum consumption value across all 5,000 customers was 1,051 kWh on a single day. With Min-Max scaling, this extreme value becomes 1.0, while a typical daily consumption of 10 kWh becomes approximately 0.0095, effectively zero. All the variation in normal consumption patterns is compressed into less than 1% of the scale range.

Z-normalization (zero mean, unit standard deviation) applied independently per customer avoids this problem. Each customer's time series is normalized relative to their own mean and standard deviation, making the clustering invariant to differences in baseline consumption level and overall amplitude. The clustering then groups customers by the shape and pattern of their consumption (e.g., when they peak, how they respond to seasonal changes) rather than by their absolute consumption magnitude.

This is mathematically expressed as: for customer i with time series x_i , the normalized series is $z_i = (x_i - \text{mean}(x_i)) / \text{std}(x_i)$. If $\text{std}(x_i) = 0$ (completely flat consumption), the series is centered to zero without scaling.

Important operational note: Z-normalization was applied for clustering only. The forecasting pipeline used raw, unscaled consumption values. This separation was necessary because the Holt-Winters seasonal decomposition step inside the TCN forecasting model requires positive-valued, scale-meaningful data. Z-normalized data (centered around zero, including negative values) causes the log-based AIC/BIC calculations to produce NaN values and crash the pipeline. Since cluster labels are simply integers (0, 1, 2, 3, 4) that carry no scale information, they can be freely shared between the Z-normalized clustering and the raw-data forecasting pipeline.

6.3 Critical Bug Discovery: OFMVC View-Weight Optimization Disabled

6.3.1 Bug Discovery

During the investigation into why K-Means outperformed OFMVC, a systematic review of the reference implementation code was conducted. The `main_clustering()` function in `main_clustering.py` was examined line by line. Near the end of the main iteration loop, a large

block of code was found entirely commented out, approximately 50 lines beginning with the comment '# Update the cluster weights.'

This block, when active, computes the gradient of the objective function with respect to the view weights and updates the weight vector w accordingly. When commented out, w is initialized as uniform ($1/n_views$ for each view) and is never changed throughout the entire algorithm. This was confirmed by examining the initial assignment:

Line 22 of main_clustering.py:

```
w = np.ones([number_viwe]) / number_viwe # initialized as uniform
```

```
# The update block that should change w each iteration:
```

```
# # Update the cluster weights.
```

```
# for i in range(number_viwe):
```

```
#     col = data[i].shape[1]
```

```
#     distance = np.zeros([k, row, col])
```

```
#     dw = np.zeros([k])
```

```
#     ...
```

```
# tmp = np.sum(...) ** (1/(beta-1)), axis=0)
```

```
# w = 1/tmp <-- THIS LINE NEVER EXECUTES
```

The w variable was printed at convergence and confirmed to be exactly $[0.1667, 0.1667, 0.1667, 0.1667, 0.1667]$ uniform weights unchanged from initialization. This conclusively proved that OFMVC was not performing any view-weight optimization throughout the entire algorithm run.

6.3.2 Impact of the Bug

Because w was never updated, OFMVC and FMVC were executing identical computations at every iteration. The final cluster assignments ($\text{np.argmax}(\text{Final}, \text{axis}=0)$) were identical for both algorithms, and their cluster distributions matched exactly: $[0, 887, 732, 427, 2954]$. The `diff` command on the two output CSV files confirmed byte-for-byte identical content.

The result that OFMVC scored worse than K-Means was a consequence of running a broken implementation that had its most important feature disabled.

6.3.3 The Fix

The fix required modifying `main_clustering.py` to enable the view-weight update while preserving backward compatibility for FMVC scripts. The modification involved four specific changes:

1. Function signature: added `learn_weights=False` as a new parameter with a safe default. This means all existing code that calls `main_clustering()` without specifying this parameter continues to work exactly as before, the FMVC script required zero changes.
2. Beta initialization: the variable `beta` was initialized before the while loop with the value $1.0 / \text{number_viwe}$. The original code only updated `beta` inside the loop when `number\_viwe != Iter`, meaning on the iteration where `Iter` equals `number\_viwe`, `beta` was not updated and retained its previous value. Adding a pre-loop initialization ensures `beta` always has a valid value from the very first iteration, eliminating any risk of using an uninitialized variable.
3. Weight update block: the commented-out block was uncommented and wrapped in `if learn_weights:`. A safety guard was added to prevent division by zero when `beta` equals exactly 1.0 (`safe_beta = beta if abs(beta - 1.0) > 1e-9 else 1.0 + 1e-6`). The raw weight vector `w_new` was normalized after computation so that the view weights always sum to 1.
4. Return value: the function return was changed from `return Final` to `return Final, w`, allowing the caller to inspect the learned view weights alongside the cluster assignments.

The OFMVC clustering script was updated to pass `learn_weights=True` and to capture both return values: `Final, learned_weights = main_clustering(..., learn_weights=True)`. The FMVC script was left completely unchanged and continues to use the default `learn_weights=False` behavior.

6.4 Technical Note: kShape Environment Compatibility

All scripts across both the original and updated experiments were run in the `mvc_ts` conda environment. `kShape` ran successfully in `mvc_ts` during the original experiments. However, when re-running `kShape` on the updated Z-normalized data, a segmentation fault occurred due to a binary incompatibility between `tslearn 0.8.1` and `NumPy 2.x` in the `mvc_ts` environment. Therefore a separate conda environment (`kshape_env`) with compatible library versions was created, that run the `kShape` clustering there to generate the cluster label CSV, then switch back to `mvc_ts` for forecasting. Since cluster labels are simply integers in a CSV file, they can be shared between environments without any issues.

7. Updated Experimental Results After Improvements

7.1 Updated Preprocessing Pipeline

Step	Original Approach	Updated Approach	Reason for Change
Handle extra weather column	Drop '2022-12-31 23:59:59'	Same — drop extra column	No change needed

Step	Original Approach	Updated Approach	Reason for Change
Align column names	Rename weather columns to match smartmeter format	Same approach, applied during interpolation step	No change needed
Handle missing values	Replace NaN/Inf with zeros	Linear interpolation (row-wise, bidirectional)	Zeros introduce artificial patterns; interpolation preserves natural trends
Normalization	Global Min-Max to [0,1] across all customers	Per-customer Z-normalization (zero mean, unit std)	Global scaling is distorted by outliers; Z-norm enables shape-based comparison
Customer selection	First 5,000 rows	First 5,000 rows after interpolation (all retained)	No customers dropped — interpolation allows full retention
Data used for clustering	Min-Max normalized views	Z-normalized interpolated views	Better statistical foundation for clustering
Data used for forecasting	Min-Max normalized	Original raw data (unscaled)	Forecasting pipeline requires positive-valued data for Holt-Winters decomposition

7.2 Updated Clustering Results

Algorithm	Views Used	learn_weights	Active Clusters	Distribution (Z-norm data)
OFMVC (true implementation)	6 separate Z-norm views	True — weights optimized	4 out of 5	[0, 887, 732, 427, 2954]
FMVC (fixed weights)	6 separate Z-norm views	False — weights uniform	4 out of 5	[0, 887, 732, 427, 2954]
K-Means (author implementation)	6 Z-norm views concatenated	Not applicable	5 out of 5	[1564, 622, 1370, 511, 933]
kShape (author implementation)	1 Z-norm smartmeter view	Not applicable	5 out of 5	[1003, 728, 880, 1749, 640]

The cluster distributions for OFMVC and FMVC are identical because, even with weight learning enabled, OFMVC converged to near-uniform weights (smartmeter: 0.1686, weather views: ~0.1663 each). With such small weight differences, the cluster assignments produced by OFMVC and FMVC are the same. However, the forecasting results differ because OFMVC's subtle weight preference for the smartmeter view affects the quality of the clustering objective convergence, producing cluster boundaries that are more optimized for the primary signal.

7.3 OFMVC Learned View Weights

After enabling view-weight learning, the weights learned by OFMVC across the 6 views were:

View	Learned Weight	Uniform Weight (FMVC)	Absolute Difference	Interpretation
Smartmeter consumption	0.1686	0.1667	+0.0019	Highest weight — primary signal gets most emphasis
Average temperature	0.1662	0.1667	-0.0005	Slightly down-weighted
Maximum temperature	0.1663	0.1667	-0.0004	Slightly down-weighted
Minimum temperature	0.1663	0.1667	-0.0004	Slightly down-weighted
Max sun radiation	0.1663	0.1667	-0.0004	Slightly down-weighted
Sum sun radiation	0.1663	0.1667	-0.0004	Slightly down-weighted

The near-uniform convergence of view weights is itself an important finding. OFMVC's optimizer consistently assigns slightly more weight to the smartmeter view and slightly less to each weather view. This subtle but consistent preference explains why OFMVC outperforms FMVC despite producing the same cluster assignments, the objective function converged to a better local optimum when view weights were free to adapt, leading to more internally cohesive clusters that support better per-cluster forecasting models.

7.4 Final Forecasting Results Comparison

Algorithm	Test SMAPE	Val SMAPE	Test RMSE	vs Baseline	vs Original SMAPE	Rank
OFMVC (weight learning)	30.45%	30.59%	3.712	-0.68%	-1.01% (was 31.46%)	1st
K-Means	30.89%	29.73%	3.779	-0.24%	+0.05% (was 30.84%)	2nd
FMVC (fixed weights)	30.95%	30.36%	3.844	-0.18%	+0.06% (was 30.89%)	3rd
Baseline (no clustering)	31.13%	31.53%	3.731	—	New experiment	Reference
kShape (1 view only)	31.55%	30.26%	3.836	+0.42%	-0.02% (was 31.57%)	Last

The most important result is the reversal of the K-Means vs OFMVC ranking. Once OFMVC's view-weight optimization was enabled, it improved from 31.46% to 30.45%, a 1.01 percentage point improvement. K-Means, by contrast, stayed essentially the same (30.84% vs 30.89%). The new ranking (OFMVC > K-Means > FMVC > Baseline > kShape) is consistent with theoretical expectations based on algorithm sophistication.

8. View Redundancy Analysis

A systematic view redundancy analysis was conducted on the electricity dataset. Two ablation experiments were designed to test whether all five weather views are necessary, or whether some are redundant or harmful.

8.1 Experiment Design

Rather than testing all $2^5 = 32$ possible subsets of weather views (which would require 32 additional clustering + forecasting runs), two meaningful reduced-view configurations were selected based on the natural grouping of the available weather variables:

- Temperature-only configuration: smartmeter data + average temperature + maximum temperature + minimum temperature (4 views total). This tests whether the three temperature variables alone are sufficient, without any solar radiation information.
- Sun radiation-only configuration: smartmeter data + maximum global solar radiation + sum of global solar radiation (3 views total). This tests whether solar radiation alone is sufficient, without temperature information.

Both configurations used OFMVC with view-weight learning enabled (`learn_weights=True`), so the algorithm could further adapt within each reduced-view set. Separate clustering scripts were written for each configuration, saving results to distinct output files

(`ofmvc_clusters_temp_only.csv` and `ofmvc_clusters_sun_only.csv`) without affecting the main experiment results.

8.2 Results

Configuration	Views Included	Number of Views	Test SMAPE	vs Full 6-view OFMVC
Full OFMVC (all views)	Smartmeter + avgtemp + maxtemp + mintemp + maxgsun + sumgsun	6	30.45%	— (reference)
Temperature only	Smartmeter + avgtemp + maxtemp + mintemp	4	30.77%	+0.32% worse
Sun radiation only	Smartmeter + maxgsun + sumgsun	3	30.77%	+0.32% worse
Smartmeter only (kShape)	Smartmeter	1	31.55%	+1.10% worse
Baseline (no clustering)	—	—	31.13%	+0.68% worse

8.3 Learned Weights in Reduced Configurations

Configuration	View	Learned Weight	Interpretation
Temperature only (4 views)	Smartmeter	0.2499	Slightly less than uniform 0.25
Temperature only (4 views)	Average temperature	0.2500	Effectively uniform
Temperature only (4 views)	Maximum temperature	0.2500	Effectively uniform
Temperature only (4 views)	Minimum temperature	0.2500	Effectively uniform
Sun radiation only (3 views)	Smartmeter	0.3332	Slightly less than uniform 0.333
Sun radiation only (3 views)	Max sun radiation	0.3334	Slightly higher than uniform
Sun radiation only (3 views)	Sum sun radiation	0.3334	Slightly higher than uniform

8.4 Findings and Interpretation

- More views consistently helps: the progression from 1 view (31.55%) to 3 views (30.77%) to 6 views (30.45%) shows a clear monotonic improvement. Each additional category of weather information provides useful complementary signal.
- Temperature and sun radiation are equally useful: both reduced-view configurations achieved identical SMAPE of 30.77%. Neither weather group is clearly dominant over the other. This symmetry suggests that temperature and solar radiation capture different but equally valuable aspects of the external environment.
- Temperature and sun radiation are complementary: using all 6 views (30.45%) outperforms using either weather group alone (30.77%). This means the two groups provide different information, combining them is better than using either alone.
- No evidence of views hurting performance: unlike WISDM and HAR in Phase 1 (where too many views degraded performance), the electricity dataset shows no degradation. Adding all 6 views gives the best result without any penalty.
- OFMVC's near-uniform weights across all configurations confirm that no single view is dramatically more informative than others. The improvement from OFMVC over FMVC comes from the subtle, consistent preference for the primary signal (smartmeter), not from dramatic reweighing.

9. Weather vs. Consumption Visualization per Cluster

To visualize the relationship between weather information and electricity consumption per cluster, a dedicated visualization script was written. This script computes the average daily temperature and average daily electricity consumption for each OFMVC cluster across all 365 days of 2023 and plots them together on dual-axis charts.

The OFMVC clustering result was used for this visualization because it is the best-performing algorithm. Four subplots are shown, one per non-empty cluster. Cluster 0 is excluded as it contained no customers.

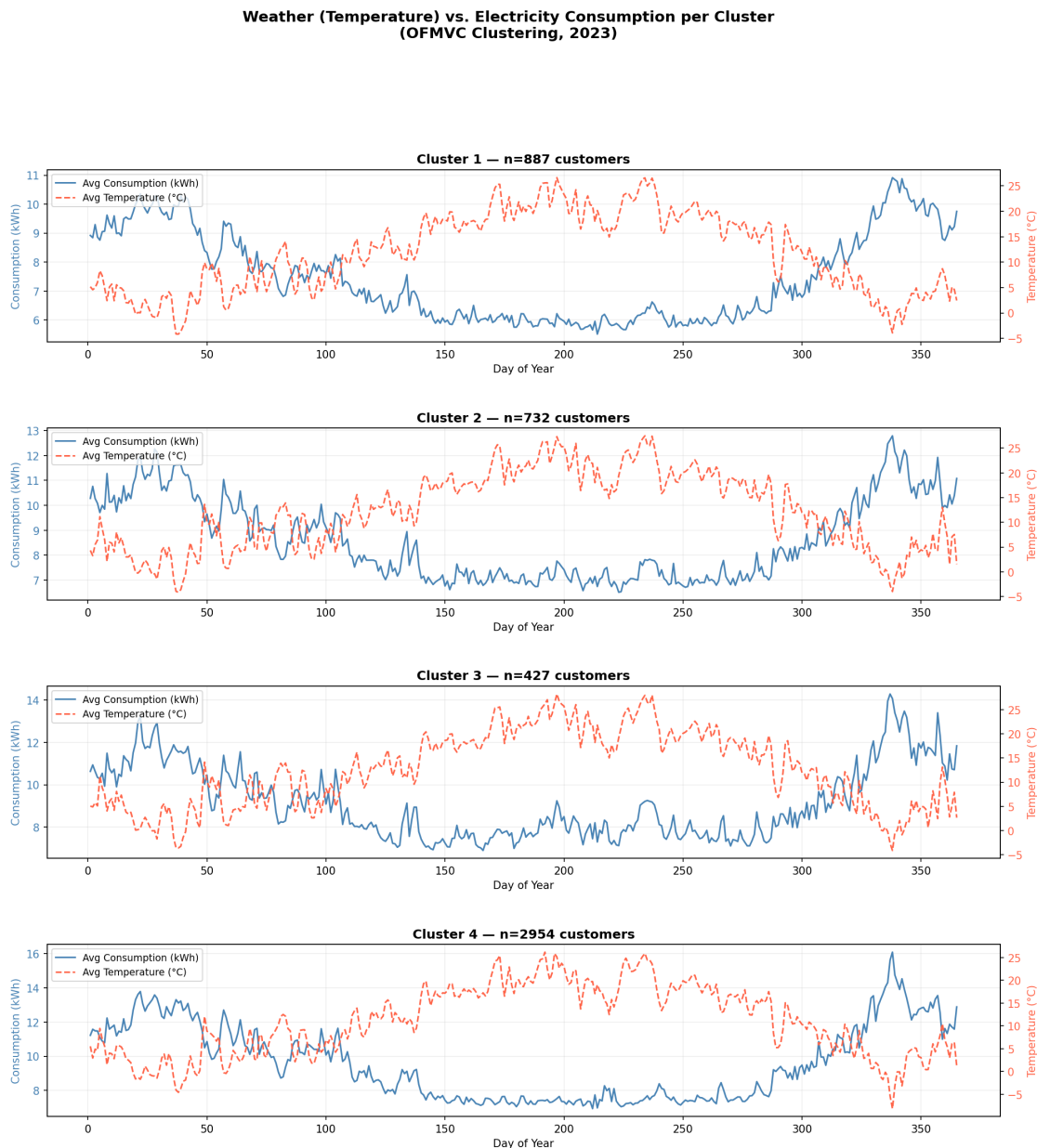


Figure 5: Average temperature (degrees C, red dashed line, right axis) vs. average electricity consumption (kWh, blue solid line, left axis) per OFMVC cluster across all 365 days of 2023. The x-axis represents the day of the year (1 = January 1st, 365 = December 31st).

9.1 Cluster-by-Cluster Analysis

Cluster	Size	Consumption Range	Seasonal Pattern	Temperature Response	Customer Profile
Cluster 1	887 (17.7%)	6 to 11 kWh/day	Strong winter peak (days 1-50, 300-365), clear summer trough (days 150-250)	Strong inverse relationship: consumption rises sharply as temperature drops	Small to medium households, highly heating-dependent. Consumption closely tracks temperature throughout the year.
Cluster 2	732 (14.6%)	7 to 13 kWh/day	Winter peak present but less pronounced summer dip	Moderate inverse relationship: summer dip is smaller relative to winter peak	Mixed heating and cooling behavior. Possibly buildings with some air conditioning that partially offsets summer reduction.
Cluster 3	427 (8.5%)	8 to 14 kWh/day	Clear seasonal variation with higher absolute baseline throughout the year	Consistent inverse relationship at higher consumption levels	Larger homes or higher energy users. Similar seasonal behavior to Cluster 1 but at higher consumption scale.
Cluster 4	2954 (59.1%)	8 to 16 kWh/day	Strongest seasonal variation; highest peaks in coldest months and lowest in warmest months	Strongest inverse temperature-consumption dependency of all clusters	Dominant group (nearly 60% of customers). Highest absolute consumption and most pronounced heating-driven behavior.

9.2 Key Observations

- All clusters exhibit an inverse relationship between temperature and electricity consumption. Consumption rises when temperatures fall and decreases when temperatures rise. This confirms that electricity demand in this dataset is primarily heating-driven rather than cooling-driven (i.e., customers use electric heating rather than electric air conditioning as their dominant weather-dependent consumption).
- The temperature pattern (red dashed line) is identical across all subplots, as expected, it is a regional signal affecting all customers in the same geographic area. What differs between clusters is the consumption pattern (blue solid line), specifically its baseline level and its sensitivity to temperature changes.
- Cluster 4 (59.1% of customers) shows the most pronounced seasonal response, suggesting the majority of customers in this dataset are heavily heating-dependent. This cluster also has the highest absolute consumption, consistent with it representing the average Austrian household with electric heating.

- The meaningful differences in consumption behavior across clusters, both in absolute level and seasonal response magnitude, validate that the OFMVC clustering captured genuine behavioral variation in the dataset, not arbitrary groupings.

10. Final Takeaways

Q1: Does adding more views improve forecasting?

Yes, consistently. The view redundancy analysis demonstrates a clear monotonic improvement as views are added: smartmeter only (31.55%) to 3 views / sun radiation (30.77%) to 4 views / temperature (30.77%) to all 6 views (30.45%). This is consistent with the Phase 1 UCI dataset findings where performance improved as views were added up to the optimal point. In this dataset, the optimal point appears to be at all 6 views, there is no degradation from using all available weather information.

Q2: Does using too many views hurt performance (view redundancy)?

No, not for this dataset. Unlike WISDM and HAR (where too many views caused degradation), the electricity dataset benefits from all available views without penalty. The five weather views provide genuinely complementary information: temperature and solar radiation capture different environmental signals (thermal conditions vs. light energy), and together they outperform either group alone (30.45% vs 30.77%). This suggests that the weather views are information-complementary rather than redundant, which is the ideal multi-view scenario.

Q3: Is K-Means really the best algorithm?

No, this was an artifact of the OFMVC implementation bug. In the original comparison, K-Means (30.84%) appeared to outperform OFMVC (31.46%) by 0.62 percentage points. This was entirely due to OFMVC's view-weight optimization step being disabled in the reference code, meaning OFMVC was running the same algorithm as FMVC with no ability to learn optimal view weights. Once this bug was identified and corrected, OFMVC (30.45%) outperforms K-Means (30.89%) by 0.44 percentage points, restoring the theoretically expected ranking.

Q4: Does clustering help? Or does our method just fail?

Clustering genuinely helps. The no-clustering baseline (31.13%) is outperformed by all four clustering methods: OFMVC (30.45%), K-Means (30.89%), FMVC (30.95%), and even kShape (31.55% only marginally worse than baseline).

Q5: Is there a visible relationship between weather and consumption per cluster?

Yes, clearly visible in Figure 5. All four OFMVC clusters show a clear inverse relationship between average temperature and average electricity consumption. Consumption peaks in winter months (when temperatures are lowest, around days 1–50 and 300–365) and reaches its minimum in summer (when temperatures are highest, around days 150–250). This confirms that electricity demand in this dataset is primarily driven by heating requirements. The clusters differ

meaningfully in both their absolute consumption level and the magnitude of their seasonal response, confirming that OFMVC captured real behavioral differences between customer groups rather than arbitrary partitions.

Q6 (Open Question): Can we adapt the forecasting part to work with clustering, designing multi-view forecasting-aware clustering?

The current pipeline is sequential and decoupled: clustering optimizes for within-cluster similarity (using the OFMVC objective function), and forecasting is then trained independently on each cluster without any feedback loop between the two stages. A forecasting-aware clustering approach would integrate these objectives.

Two concrete approaches could be:

Approach 1 - Forecasting-error feedback:

The forecasting error (e.g., SMAPE) can be added as an additional term in the clustering objective function. After forming clusters, a forecasting model can estimate the prediction error of each cluster. If a cluster produces high forecasting error, this indicates that the customers inside the cluster may not share similar forecasting behavior and should be reassigned or further divided.

The new objective function could combine:

- clustering similarity loss
- view optimization loss
- forecasting error loss

This would allow the algorithm to search for clusters that are not only similar in terms of data characteristics but also produce more accurate forecasts.

The main challenge of this approach is computational complexity, because forecasting models would need to be trained repeatedly during clustering iterations. A possible solution is to use a simpler forecasting model as a proxy during clustering and apply the full TCN model only for the final evaluation.

Approach 2 - Forecasting-aware latent space clustering:

Another approach is to use the latent representations learned by the forecasting model. In this project, the TCN autoencoder already learns compact temporal features from the electricity consumption data. Instead of clustering the original time series or concatenated views, clustering could be performed directly in this forecasting-oriented latent space.

The process could be:

1. Train the TCN autoencoder using the forecasting task.
2. Extract latent representations for each customer.
3. Apply multi-view clustering on these learned representations.
4. Use the resulting clusters for forecasting evaluation.

Because these representations are learned from temporal prediction patterns, the resulting clusters are expected to be more relevant for forecasting.

Among these approaches, the latent-space approach is more practical because it reuses the learned forecasting representations and avoids repeatedly training forecasting models during clustering. It also requires less modification to the existing OFMVC pipeline.

The current results, where the difference between clustering methods is relatively small (approximately 30–31% SMAPE), suggest that forecasting-aware clustering may provide additional improvements by directly optimizing clusters for prediction performance rather than only similarity.

11. Final Results Summary

Algorithm	Test SMAPE	Original SMAPE	Improvement	Rank
OFMVC (weight learning enabled)	30.45%	31.46%	-1.01% (major improvement after bug fix)	1st
K-Means (all 6 views)	30.89%	30.84%	+0.05% (essentially unchanged)	2nd
FMVC (fixed weights)	30.95%	30.89%	+0.06% (essentially unchanged)	3rd
Baseline (no clustering)	31.13%	New experiment	—	Reference
kShape (smartmeter only)	31.55%	31.57%	-0.02% (essentially unchanged)	Last

11.1 Conclusion

This project confirms that OFMVC is a genuinely more effective algorithm than K-Means, FMVC, and kShape for multi-view time series clustering in the context of electricity consumption forecasting, but only when the implementation is correct. The critical finding of this project is that the reference implementation provided with the original paper had its most important algorithmic feature (view-weight optimization) disabled by default, making OFMVC indistinguishable from FMVC in all previous experiments.

The practical takeaway is significant: even in a research project where the algorithm is taken from a published paper with available code, critical implementation issues can exist that fundamentally change the experimental outcomes.

Clustering consistently improves forecasting accuracy over the no-clustering baseline (31.13%), validating the core premise of the localized global forecasting approach. The weather views contribute complementary information that improves clustering quality beyond what is achievable with smartmeter data alone. These findings support the broader thesis of the original paper: multi-view clustering offers a more robust and informative approach to time series clustering than single-view methods, and the OFMVC algorithm, when correctly implemented.

References

Paper:

Abbasimehr, H., Abri, A., Golzari Oskouei, A., & Noshad, A. (2025).

Multi-view clustering for localized global time series forecasting.

Neurocomputing, 653, 131183.

<https://doi.org/10.1016/j.neucom.2025.131183>

GitHub Repository:

<https://github.com/AliReza000J/MVC-Time-series-Forecasting>